

Лабораторна робота №2.

Вивчення системи введення/виведення мікроконтролерів PSoC 6.

Мета роботи: Ознайомитися з системою введення/виведення (General Purpose Input/Output – GPIO) мікроконтролерів PSoC 6; методами налаштування, читання, запису виводів портів мікроконтролера; реалізувати проекти з використанням GPIO мікроконтролера PSoC 6.

Теоретичні відомості.

Система введення/виведення забезпечує гнучкий інтерфейс між ядром центрального процесора (ЦП) та периферійними компонентами для зовнішнього світу. Гнучкість мікроконтролерів PSoC 6 і можливість його виводів GPIO по введенню/виведенню даних значно спрощує проектування схем та компоновку компонент схеми на платі. Виводи GPIO в сімействі мікроконтролерів PSoC 6 об'єднані в порти. Порт може мати максимум вісім виводів GPIO.

Основні характеристики GPIO мікроконтролерів PSoC 6:

- можливість введення і виведення аналогових та цифрових сигналів;
- вісім режимів роботи виводів портів;
- окремі регістри для читання порту і запису в порт;
- виводи з захистом від допустимих перевищень напруги (OVT-GPIO);
- окремі джерела введення/виведення і напруг для груп введення/виведення (не більше шести);
- можливі переривання по передньому фронту, по задньому фронту або по обох фронтах на всіх виводах GPIO;
- контроль швидкості наростання фронтів сигналів на виводах GPIO;
- режим утримання для фіксації попереднього стану (використовується для зберігання стану введення/виведення в режимі глибокого сну);
- вибір режиму CMOS або низьковольтного входного буферу з LVTTTL;
- підтримка сенсору дотику (CapSense);
- можливість виконувати логічні функції в колі введення/виведення з інтелектуальними виводами;
- підтримка сегменту LCD пристрою.

Загальний опис компоненти GPIO.

Компонента GPIO - це графічний об'єкт конфігурації, побудований поверх драйвера `su_gpio`, доступного в бібліотеці периферійних драйверів (Peripheral Driver Library – PDL). Вона дозволяє з'єднання на основі схеми та апаратну конфігурацію, як це визначено в діалоговому вікні "Component Configure".

Компонента GPIO дозволяє апаратним ресурсам під'єднатися до фізичного виводу порту. Вона забезпечує доступ до зовнішніх сигналів через правильно

налаштований фізичний вивід введення/виведення. Також вона дозволяє вибирати електричні характеристики (наприклад, режим драйвера) для одного або декількох виводів. Ці характеристики потім використовуються середовищем розробки PSoC Creator 4.4 для автоматичного розміщення та маршрутизації сигналів в компоненті.

Компоненту GPIO можна використовувати із з'єднанням провідників зі схемою, програмним забезпеченням або обома. Вона може бути налаштована на багато типів комбінацій. *Каталог компонентів містить чотири попередньо налаштовані компоненти GPIO: аналогову, цифрову двонаправлену, цифровий вхід та цифровий вихід.*

Компонента GPIO використовується, коли в проєкті необхідно генерувати або отримувати доступ до сигналу поза мікроконтролером через фізичний вивід введення/виведення. Це зручний спосіб налаштовувати виводи в графічному вигляді і дозволяє PSoC Creator IDE виконувати маршрутизацію та мультиплексування виводів.

Відображення GPIOs можна налаштувати на складні комбінації цифрового входу, цифрового виходу, цифрового двонаправленого та аналогового виводів. Прості конфігурації, як правило, відображаються у вигляді окремих виводів. Більш складні типи виводів показані як стандартні компоненти з обмежувальною рамкою.

Параметри компонентів.

Для того, щоб відкрити діалогове вікно Configure потрібно перетягнути компоненту GPIO на схему проєкту та двічі клацнути на ній лівою кнопкою мишки (рис. 2.1). Це вікно використовується для задання параметрів компонент та організоване у вигляді окремих вкладок.

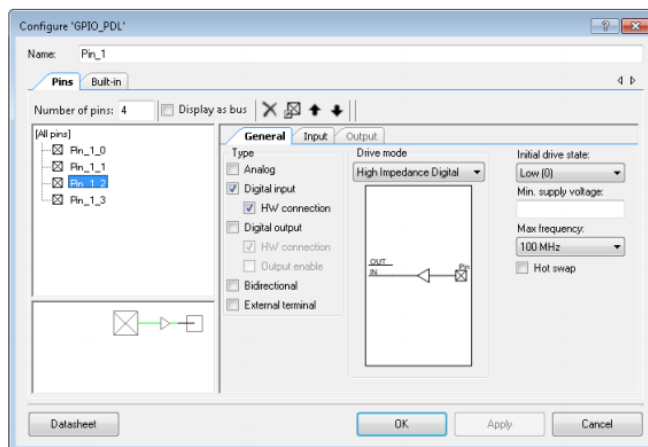


Рис. 2.1. Вигляд вікна налаштування виводів GPIO_PDL

Вкладка *Виводи* (Pins) має три області: панель інструментів, дерево виводів і набір вкладених вкладок. Панель інструментів використовується для задання кількості фізичних виводів компоненти і визначення їх порядку. Вкладені

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж” 2025 р. Рабик В.

вкладки використовуються для задання специфічних для виводів атрибутів, таких як тип виводів, режим драйвера і початковий стан.

Панель інструментів містить наступні команди:

- *Число виводів.* (Кількість виводів пристрою, які контролюються компонентом. За замовчуванням – 1 вивід.)
- *Відображення шини.*

Цей параметр вибирає, чи відображати окремі термінали для кожного виводу або один широкий термінал (шину). Варіант шини дійсний лише тоді, коли виводи однорідні. Це означає, що всі виводи компоненти мають однаковий тип, вхідні/вихідні з'єднання HW та групування SIO (special I/O – SIO). Всі вони також повинні використовувати або не використовувати SIO Vref.

- *Видалення виводу.* (Видаляє вибрані виводи з дерева).
- *Додати/змінити ім'я.* (Відкриває діалогове вікно для додавання або зміни імені для вибраного виводу в дереві. Також можна відкрити діалогове вікно двічі клацнувши на виводі або натиснувши [F2]).
- *Перемістити вгору/вниз.* (Переміщує вибрані виводи вгору або вниз по дереву).
- *З'єднувати/ не з'єднувати.* (З'єднує або скасовує вибрані SIO виводи позначені рожевим контуром у дереві).

Цей елемент керування визначає, чи потрібно виводи, які потребують, розміщувати в одній і тій же парі SIO на пристрої. Узгодження виводів призводить до того, що менше фізичних SIO виводів "витрачається". Щоб виводи мали спільну пару SIO на пристрої, вони повинні мати однакові налаштування для кожної пари і бути суміжними.

Для виводу потрібно SIO, якщо вибрано функцію "Гаряча заміна" (Hot Swap), для параметру "Поріг" (Threshold) задано будь-яке значення крім LVTTTL або CMOS, для рівня драйвера встановлено значення "Vref" і/або для параметру "Струм" встановлено значення "25 мА споживання".

В області "Дерево виводів" (Pin Tree) відображаються всі виводи компоненти. Можна індивідуально вибрати один або декілька виводів для використання з командами панелі інструментів і вкладеними вкладками.

Кожний вивід відображає своє ім'я, яке складається з <Імені екземпляра GPIO>_<Індивідуального імені виводу>.

Під деревом знаходиться область попереднього перегляду, яка показує який вигляд матиме вибраний символ компоненти GPIO з різними параметрами, вибраними для цього конкретного виводу.

Підкладка "Загальне" (General). Ця підкладка за замовчуванням, що відображається на вкладці "Виводи", зображена на рис 2.2.

Вона містить такі параметри:

На ній вибираємо тип виводів для компонента за допомогою прапорців.

- *Аналоговий.* Вибираємо "Аналоговий", щоб увімкнути термінал аналогового виводу. Це дозволяє маршрутизацію аналогового сигналу до інших компонент.

Вибір аналогового виводу змушує вивід фізично розміщуватися на виводі GPIO, а не на виводі SIO.

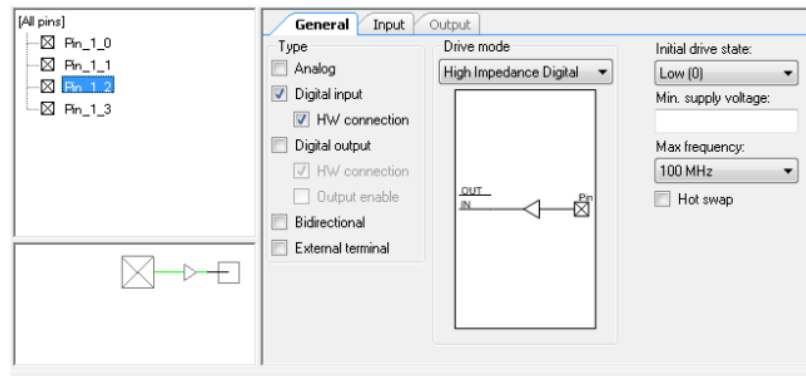


Рис. 2.2. Вигляд підвкладки "Загальне" вкладки "Виводи"

- *Цифровий вхід*. Вибираємо "Цифровий вхід", щоб увімкнути термінал цифрового входу для введення (необов'язково) та включити вкладку "Вхід" для додаткових параметрів конфігурації, пов'язаних з входами.

- *HardWare (HW) з'єднання*. Цей параметр визначає чи буде цифровий вхідний термінал для вхідного виводу відображатися на схемі. Якщо відображається, то вивід забезпечує цифровий сигнал, який може під'єднуватися до інших цифрових приймачів, таких як термінали компонентів. Якщо ця опція не вибрана, термінал не відображається і може бути прочитаний тільки CPU/DMA.

- *Цифровий вихід*. Вибираємо "Цифровий вихід", щоб увімкнути термінал цифрового виходу (необов'язково) та включити підвкладку "Вихід" для додаткових параметрів конфігурації, пов'язаних з виходами.

- *Output Enable* (включення виходу). Цей параметр дозволяє використовувати функцію включення виходу для виводу і відображає вхідний термінал включення виводу. Функція включення виходу дозволяє апаратному (DSI) сигналу керувати драйверами виходів виводу без ЦП. Рівень логічної "1" конфігурує драйвери виходу як встановлено в параметрі "Режимів драйвера" (Drive Mode). Низький логічний рівень від'єднує драйвери виходу і переводить вивід в режим високоомного драйвера.

- *Двонаправлений вивід*. Дозвіл двонаправленого параметра функціонально еквівалентний включенню цифрового входу з під'єднанням HW і цифрового виходу з параметром під'єднання HW. Різниця полягає в тому, що на символі компоненти відображається тільки один двонаправлений термінал, а не окремі вхідні та вихідні виводи. Потім термінал може під'єднуватися до інших з'єднань двонаправленого типу. Як вхідна, так і вихідна вкладені вкладки включені для подальшого налаштування.

- *Зовнішній термінал (External Terminal)*. Дозволяє з'єднання з зовнішніми компонентами з каталогу компонент для ілюстрації електричної схеми поза мікроконтролером.

Режим драйвера (Drive mode). Цей параметр налаштовує вивід для забезпечення одного з 8-ми доступних режимів драйвера виводу. Значення за замовчуванням і логічний вибір залежать від вибору типу. Діаграма показує представлення схеми для кожного вибраного режиму драйвера.

- Якщо тип є цифровим входом або аналоговим входом за замовчуванням використовується високоімпедансний цифровий вхід.

- Якщо тип є аналоговим, за замовчуванням використовується аналоговий високоімпедансний вхід. Це єдиний режим драйвера виводу, який може підтримувати тільки аналогові виводи.

- Якщо тип є двонаправленим виводом або двонаправленим/аналоговим, за замовчуванням встановлено "Відкритий стік", "Низький рівень драйвера".

- Всі інші типи виводу за замовчуванням встановлені в "Сильний драйвер".

Ініціалізація стану драйвера. Цей параметр вказує специфічне для виводу початкове значення, записане в регістр OUT виводів після скидання/включення пристрою. Це відбувається під час процесу налаштування порту в коді запуску пристрою. Якщо не змінювати вручну або автоматично налаштувати в стан логічної "1" з допомогою параметру "Режим драйвера", всі виводи за замовчуванням мають рівень логічного "0".

Конфігурація виводів GPIO мікроконтролера PSoC 6.

Інструменти конфігурації пристрою, такі як PSoC Creator, автоматично генерують код конфігурації GPIO та виконують його як частину процесу завантаження пристрою в `cyfitter_cfg.c`. Методи конфігурації GPIO, як правило, використовуються лише з ручним налаштуванням PDL GPIO, коли не використовується інструмент конфігурації. Вони також можуть використовуватися під час роботи для динамічного переналаштування виводів GPIO незалежно від того, як виконувалася початкова конфігурація.

Більшість виводів GPIO вимагають встановлення лише своїх основних параметрів і можуть використовувати значення за замовчуванням для всіх інших параметрів. Це дозволяє використовувати спрощену функцію ініціалізації.

Функція `Cy_GPIO_Pin_FastInit()` підтримує лише параметризовану конфігурацію режиму пристрою, рівня логічного виходу та налаштування швидкісного мультиплексора введення/виведення (HSIOM). Налаштування HSIOM визначає програмне забезпечення високого рівня, периферійне та аналогове управління та підключення. Усі інші налаштування конфігурації не змінюються після їх скидання або раніше встановленого стану. Ця функція дуже корисна під час виконання, щоб динамічно змінювати конфігурацію виводу. Наприклад, налаштуємо вивід в режим сильного драйвера для запису даних, а потім переналаштуємо вивід в режим високого імпедансу для зчитування даних.

```
Cy_GPIO_Pin_FastInit(P0_3_PORT, P0_3_NUM, CY_GPIO_DM_STRONG, 1, HSIOM_SEL_GPIO);
```


Метод налаштування всіх атрибутів одного виводу полягає у використанні функції `Cy_GPIO_Pin_Init()` та структури конфігурації виводів. Хоча він простий у використанні, він генерує більший код, ніж інші методи конфігурації.

```
Cy_GPIO_Pin_Init(P0_4_PORT, P0_4_NUM, &P0_4_Pin_Init);
```

Найбільш ефективним методом для налаштування всіх атрибутів для повного порту виводів є використання функції `Cy_GPIO_Port_Init()` та структури конфігурації порту. Він упакує всі дані конфігурації в прямі записи в регістр для всього порту. Його обмеження полягає в тому, що він повинен налаштувати всі виводи порта, і користувач повинен обчислити об'єднані значення регістрів для всіх виводів або скопіювати їх з інструмента конфігурації. Цей метод, застосовується інструментами автоматичної конфігурації як частина процесу завантаження пристрою в `cyfitter_cfg.c`.

```
Cy_GPIO_Port_Init(GPIO_PRT5, &port5_Init);
```

Методи читання даних з виводу GPIO.

Існують декілька методів зчитування даних з виводу GPIO. Тому для кожного конкретного випадку використання потрібно вибрати найбільш відповідний метод. Функція `Cy_GPIO_Read()` є потоковою та багатоядерною. Більшість функцій драйверів GPIO потребують як мінімум двох аргументів для визначення порту та виводу в цьому порту. Аргумент `Port` очікує базової адреси регістрів порту. Аргумент `Pin` очікує на номер виводу цього порту.

Кращим способом зчитування для використання з інструментами конфігурації є використання `#defines`, наданих для імен виводів інструментів конфігурації. У PSoC Creator 4.4 – це компоненти виводу, розміщені на схемі, а `#defines` виводів розміщені у `\\pdl\Pins` та `Interrupts\cyfitter_gpio.h`.

```
pinReadValue = Cy_GPIO_Read(SW2_P0_4_PORT, SW2_P0_4_NUM);
```

Кращим методом зчитування стану виводів для прямого використання PDL без інструменту конфігурації є використання користувацьких імен виводів `#define pin`. Користувацькі `#defines` зазвичай розміщуються у створеному користувачем `*.h` файлі.

```
#define mySwPin_Port P0_4_PORT
```

```
#define mySwPin_Num P0_4_NUM
```

```
pinReadValue = Cy_GPIO_Read(mySwPin_Port, mySwPin_Num);
```

Також можна прочитати стан виводів, використовуючи ім'я виводу `#defines` за замовчуванням для пристрою, яке є передбаченим для кожного пристрою та пакету. За замовчуванням ім'я `#defines` знаходиться в `\\Generated_Source\\(device family)\\pdl\\devices\\(device family)\\(device series)\\include\\gpio_(series+package).h`.

```
pinReadValue = Cy_GPIO_Read(P0_4_PORT, P0_4_NUM);
```

Також підтримується читання стану виводу з використанням номера порту та номера виводу порту. Цей метод корисний для автоматично генерованих портів і виводів. `Cy_GPIO_PortToAddr()` – це допоміжна функція, яка

перетворює номер порту в потрібну базову адресу регістру порту, яка потрібна для інших функцій драйвера GPIO.

```
portNumber = 0;
```

```
pinReadValue = Cy_GPIO_Read(Cy_GPIO_PortToAddr(portNumber), 4);
```

Як і для будь-якого мікроконтролера, прямий доступ до регістру портів завжди доступний і корисний для одночасного доступу до декількох виводів порту або для розробки оптимізованого для додатків доступу до портів. В наступному прикладі показано читання регістру порту IN з маскою і зсувом потрібних даних виводу.

```
pinReadValue = (GPIO_PRT0->IN >> P0_4_NUM) & CY_GPIO_IN_MASK;
```

Методи запису даних в виводи GPIO мікроконтролера PSoC 6.

Наведені нижче методи виконують один і той ж запис в виводи GPIO, використовуючи різні доступні методи запису. Завжди вибирається найбільш відповідний метод для конкретного випадку використання. Функцію `Cy_GPIO_Write()` найкраще використовувати, коли потрібний стан виводу ще невідомий і визначається під час виконання. Функція запису використовує операції, які безпосередньо впливають лише на вибраний вивід без використання операцій читання-модифікація-запис. Таким чином, функція запису є багатотоковою та багатоядерною.

Переважаючим методом прямого використання PDL без інструменту конфігурації є запис виводів із користувацькими іменами `#define`. Надані користувачем `#defines` зазвичай розміщуються у створеному користувачем *.h файлі.

```
#define myLedPin_Port P1_5_PORT
```

```
#define myLedPin_Num P1_5_NUM
```

```
Cy_GPIO_Write(myLedPin_Port, myLedPin_Num, pinReadValue);
```

Також можна записати дані у виводи, використовуючи ім'я виводу за замовчуванням для пристрою `#defines`, розміщеного в `\\_mainapp_(device series)pdl\Includes\(..devices/series/include)gpio_(series + package).h` файлі.

```
Cy_GPIO_Write(P1_5_PORT, P1_5_NUM, pinReadValue);
```

Запис в виводи можливий з використанням імені порту за замовчуванням `#defines` і номера виводів. `#defines` знаходиться в файлі `\\_mainapp_(device series)pdl\Includes\(..devices/series/include)(part number).h`.

```
Cy_GPIO_Write(GPIO_PRT1, 5, pinReadValue);
```

Для алгоритмічно генерованих номерів портів і виводів корисними є записи в виводи з використанням номерів портів і виводів. Функція `Cy_GPIO_PortToAddr()` перетворює номер порту в потрібну базову адресу регістру порту.

```
portNumber = 1;
```

```
Cy_GPIO_Write(Cy_GPIO_PortToAddr(portNumber), 5, pinReadValue);
```

Найбільш ефективні методи виводу, коли бажаний стан виводів вже відомий під час компіляції, - це безпосередньо встановлювати, очищати та інвертувати стан виводів. Ці записи в регістр є операціями, які напряду впливають тільки на вибраний вивід без використання операцій читання-модифікації-запису. Можуть також використовуватися ті ж варіанти аргументів, які продемонстровані з допомогою функції `Cy_GPIO_Write()`.

```
Cy_GPIO_Set(KIT_LED1_PORT, KIT_LED1_NUM);  
Cy_GPIO_Clr(KIT_LED1_PORT, KIT_LED1_NUM);  
Cy_GPIO_Inv(KIT_LED1_PORT, KIT_LED1_NUM);
```

Доступ до порту.

Прямий доступ до регістру використовується для взаємодії з декількома виводами в одному порту одночасно. Ці звертання не можуть бути багатоядерними через можливі операції читання-модифікації-запису. Всі виводи в порту, які знаходяться під прямим контролем регістру, повинні бути доступними тільки одному ядру центрального процесора, якщо захист доступу не забезпечений на системному рівні.

```
portReadValue = GPIO_PRT7->IN;  
portReadValue++;  
GPIO_PRT7->OUT = portReadValue;
```

Приклад реалізації проекту з використанням GPIO.

Реалізуємо проект, який демонструє декілька методів налаштування, читання, запису виводів введення/виведення загального призначення (GPIO) мікроконтролера PSoC 6. Схема цього проекту зображена на рис. 2.3.

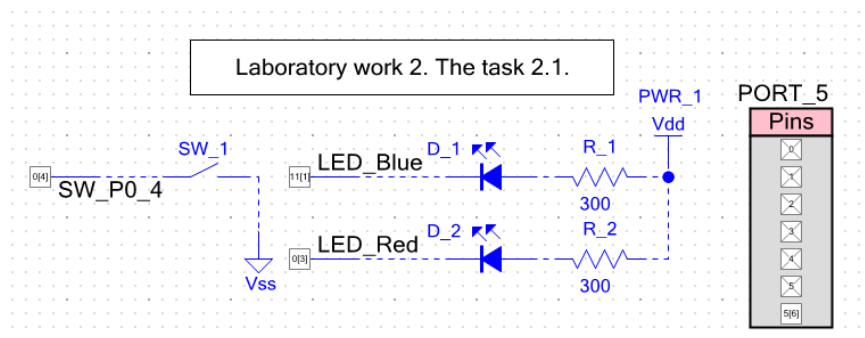


Рис. 2.3. Схема проекту для роботи з виводами GPIO PSoC 6

У цьому проекті використовується цифровий вивід GPIO, налаштований на вхід (PORT0[4]). До нього під'єднано кнопку SW2 стенду. На схемі вона позначена як SW_1. Стан кнопки постійно зчитується процесорним ядром Cortex M4. Якщо кнопка SW2 не натиснута, то вивід GPIO PORT0[4] буде в стані логічної "1". При її натисканні – в стані логічного "0". Зчитане значення стану кнопки записується в цифровий вивід LED_Red, який під'єднаний до червоного світлодіоду LED RGB стенду. Якщо кнопка SW2 не натиснута, то червоний світлодіод не світиться. При натисканні кнопки і утриманні її в натиснутому стані червоний світлодіод світитиметься.

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж” 2025 р. Рабик В.

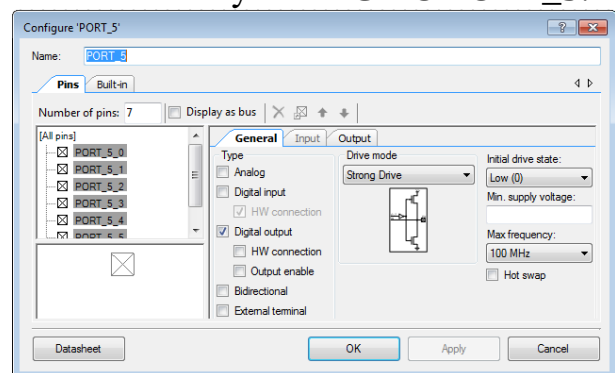
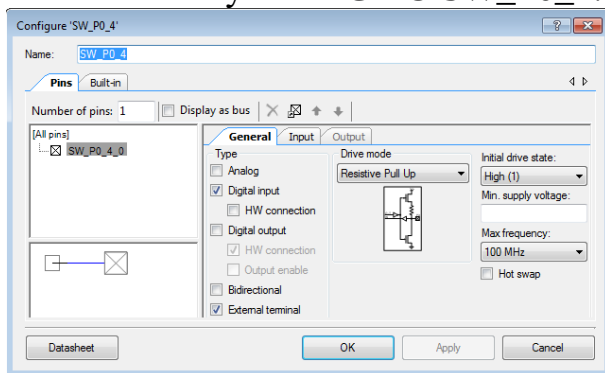
До іншого цифрового виводу LED_Blue під'єднано синій світлодіод. Його функція в цьому проекті полягає в індикації станів натискання кнопки SW2 та відпускання її, якщо вона була в натиснутому стані. Час цього засвічування малий і складає 200 мсек. Для того, щоб одночасно не світилися червоний і синій світлодіоди спочатку потрібно засвітити синій світлодіод на 200 мсек, а потім – червоний світлодіод.

Для розпізнавання моментів натискання і відпускання кнопки SW2 в цій лабораторній роботі не використовуватимемо переривання GPIO мікроконтролера.

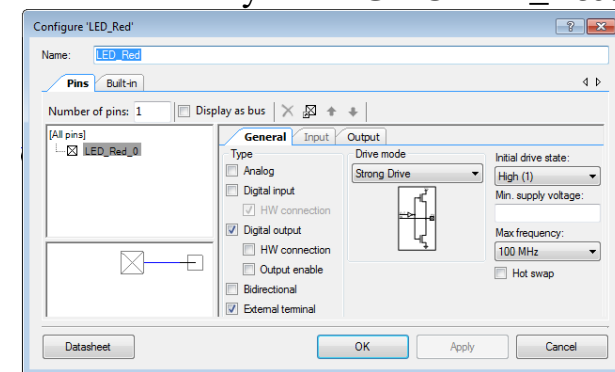
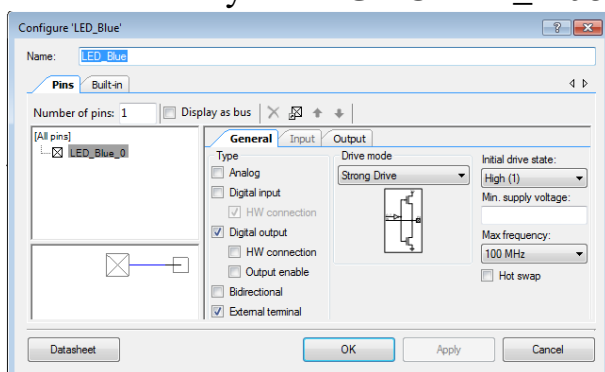
Призначення 7-ми виводів порта PORT_5 полягає в підрахунку кількості натискань на кнопку SW2. Максимальна кількість складає $2^7=128$. Після цього порт PORT_5 онулиться та підрахунок почнеться спочатку. Побачити стан порта, який не підключений до індикатора, можна з допомогою мультиметра або осцилографа.

Налаштування виводів GPIO цього проекту зображені на рисунках нижче.

Вікно налаштування GPIO SW_P0_4: Вікно налаштування GPIO PORT_5:



Вікно налаштування GPIO LED_Blue: Вікно налаштування GPIO LED_Red:



Вигляд карти виводів мікроконтролера PSoC 6 з назначеними виводами зображено на рис. 2.4.

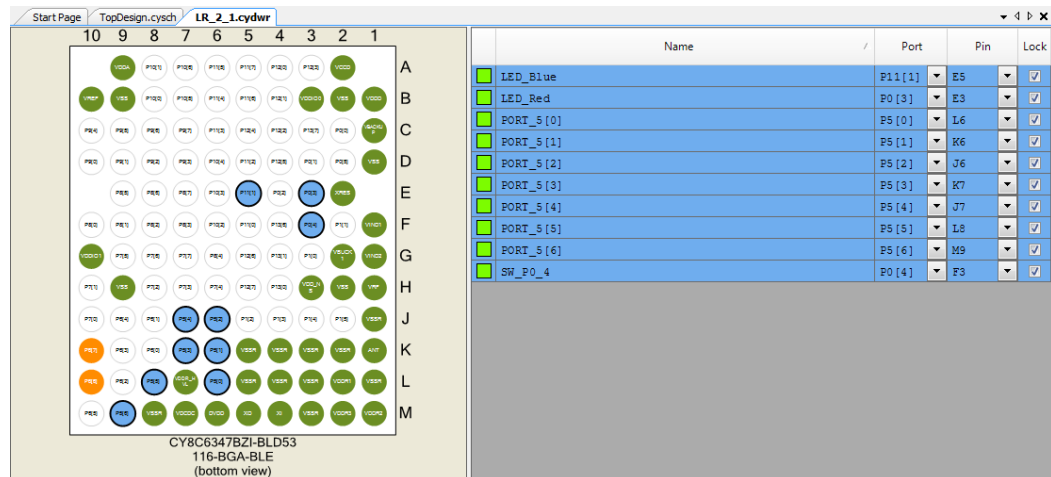


Рис. 2.4. Вигляд карти виводів мікроконтролера PSoC 6 з назначеними виводами

Деренчання контактів.

При комутації механічних контактів в різних контактних пристроях (кнопках, перемикачах, контактах реле) в початковий момент часу виникає перехідний процес. Це явище при комутації контактів називається деренчанням. Для усунення впливу деренчання контактів використовують два підходи:

- апаратний;
- програмний.

Апаратні способи реалізуються у вигляді наступних типів схем:

- тригерні схеми;
- схеми на основі RC-ланок;
- схеми на основі лічильників або зсуваючих регістрів.

Найпростіша тригерна схема для усунення деренчання контактів зображена на рис. 2.5:

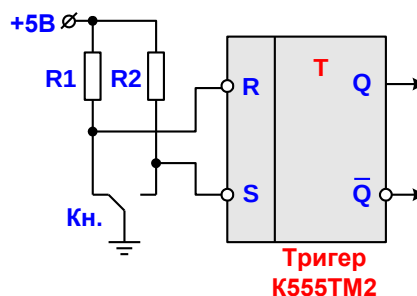


Рис. 2.5. Схема на основі тригера для усунення деренчання контактів

Для RS- тригера, зображеного на рис. 2.5, активним рівнем сигналу є логічний "0". Якщо на вхід R подати сигнал логічного "0", а на вхід S – сигнал логічної "1", то вихід тригера Q скинеться в логічний "0", а вихід $\bar{Q}$ встановиться в стан логічної "1". У випадку, якщо на R подати сигнал логічної "1", а на вхід S – сигнал логічного "0", то вихід тригера Q встановиться в стан

логічної "1", а вихід $\bar{Q}$ скинеться в стан логічного "0". При подачі на входи R і S сигналів логічної "1", RS – тригер зберігатиме свій стан. На схемі, зображеній на рис. 2.5, вихід RS – тригера переключиться тільки по першому імпульсу (логічному "0") на вході R, всі решта імпульсів, викликані деренчанням контактів, не призведуть до зміни стану на виході тригера.

Найпростіша RC – ланка призначена для усунення деренчання контактів, зображена на рис. 2.6:

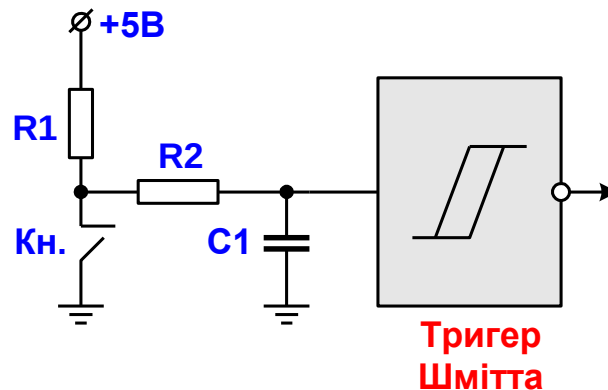


Рис. 2.6. Схема на основі RC- ланки для усунення деренчання контактів

Програмні способи усунення впливу деренчання контактів. Виділяють два основних способи:

- спосіб N-кратного зчитування;
- спосіб часової затримки.

В програмній частині цього проекту використовуватимемо для усунення впливу деренчання контактів часову затримку на 10 мсек.

Програмна частина проекту:

```
// Code LR_2_1
#include "project.h"

#define LED_ON (uint8_t) (0u)
#define LED_OFF (uint8_t) (1u)
#define LED_Blue_PORT P11_1_PORT
#define LED_Blue_Num P11_1_NUM
#define LED_Red_PORT P0_3_PORT
#define LED_Red_Num P0_3_NUM
#define SW_P0_4_PORT P0_4_PORT
#define SW_P0_4_Num P0_4_NUM

int main(void)
{
    volatile uint32_t pinReadValue = 1ul;
    uint32_t portReadValue = 0ul;

    for (;;)
    {
L1:    pinReadValue = Cy_GPIO_Read(SW_P0_4_PORT, SW_P0_4_Num);
        if (pinReadValue != 0) goto L1;
```

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж” 2025 р. Рабик В.

```
L2:   CyDelay(10);
      if (pinReadValue != 0) goto L2;
      Cy_GPIO_Write(LED_Blue_PORT, LED_Blue_NUM, LED_ON);
      CyDelay(200);
      Cy_GPIO_Write(LED_Blue_PORT, LED_Blue_NUM, LED_OFF);
      Cy_GPIO_Write(LED_Red_PORT, LED_Red_NUM, pinReadValue);

L3:   pinReadValue = Cy_GPIO_Read(SW_P0_4_PORT, SW_P0_4_NUM);
      if (pinReadValue == 0) goto L3;
L4:   CyDelay(10);
      if (pinReadValue == 0) goto L4;
      Cy_GPIO_Write(LED_Red_PORT, LED_Red_NUM, pinReadValue);
      Cy_GPIO_Write(LED_Blue_PORT, LED_Blue_NUM, LED_ON);
      CyDelay(200);
      Cy_GPIO_Write(LED_Blue_PORT, LED_Blue_NUM, LED_OFF);

      portReadValue = GPIO_PRT5->IN;
      portReadValue++;
      GPIO_PRT5->OUT = portReadValue;
      CyDelay(100);
  }
}
/* [] END OF FILE */
```

Завдання:

В звіті потрібно привести апаратну і програмну частини проекту та скріншоти налаштувань компонентів проекту.

1. Реалізувати проект, в якому послідовно засвічуються червоний, синій і зелений світлодіоди RGB LED стенду PSoC 6 BLE PIONEER KIT з використанням функцій встановлення (Cy_GPIO_Set()), скидання (Cy_GPIO_Clr()) і інвертування (Cy_GPIO_Inv()) виводів GPIO мікроконтролера PSoC 6. Час першого засвічування цих світлодіодів складає відповідно 1.75 сек, 1.5 сек та 1.25 сек. Під час наступних їх засвічувань час циклічно змінюється. Наприклад, час засвічування червоного світлодіода – 1.75 сек, 1.5 сек, 1.25 сек, 1.75 сек, ... Аналогічно час засвічування синього світлодіода - 1.5 сек, 1.25 сек, 1.75 сек, 1.5 сек ...

2. Реалізувати проект, в якому послідовно засвічуються червоний, синій і зелений світлодіоди RGB LED стенду PSoC 6 BLE PIONEER KIT з використанням функцій Cy_GPIO_PortToAddr() і Cy_GPIO_Write(). Час засвічування цих світлодіодів однаковий і задається в кожному циклі їх засвічування вектором LED_Glow_Time[1000, 750, 500, 250, 500, 750, 1500] мсек.

3. Реалізувати проект, в якому засвічується синій світлодіод RGB LED стенду PSoC 6 BLE PIONEER KIT з періодом 3.2 сек (1.8 сек – ON, 1.4 сек – OFF), якщо кнопка SW2 не натиснута. При натисканні кнопки SW2 та її

Методичні вказівки з курсу “Вбудовані системи опрацювання даних та управління на основі нейронних мереж” 2025 р. Рабик В.

утриманні засвічується зелений світлодіод RGB LED з періодом 1.8 сек (0.6 сек – ON, 1.2 сек – OFF).

4. Реалізувати проект, в якому послідовно засвічуються червоний, синій і зелений світлодіоди RGB LED стенду PSoC 6 BLE PIONEER KIT при кожному натисканні на кнопку SW2 (враховувати деренчання її контактів).